

1. Parallel Programming and Concurrency

- Concurrency means managing multiple tasks at the same time (not necessarily running simultaneously). It focuses on dealing with many things at once. Example: A web server handling multiple requests.
 - Parallelism means executing multiple tasks at the same time on different processors/cores. It's about doing many things at the same time. Example: Splitting a big array into chunks and processing each chunk on a separate CPU core.
 - **Key Tools in C#:**
 - Thread and ThreadPool for low-level concurrency.
 - Task Parallel Library (TPL) for easier parallel execution (Parallel.For, Parallel.ForEach).
 - async/await for managing asynchronous workflows.
 - **Benefit:** Improves performance and resource utilization in multi-core environments.
-

2. Unit Testing and Test-Driven Development (TDD)

- **Unit Testing:**
 - Testing individual units of code (methods, classes) in isolation.
 - Ensures correctness, prevents regression, and improves maintainability.
 - Common frameworks: NUnit, MSTest, xUnit.
 - **Test-Driven Development (TDD):**
 - A software development approach where you write tests before writing the actual code.
 - Process:
 1. Write a failing unit test (Red).
 2. Write code to make the test pass (Green).
 3. Refactor code while keeping tests passing (Refactor).
 - Benefits: Ensures reliability, drives better design, and reduces bugs early.
-

3. Asynchronous Programming with async and await

- **Purpose:** Enables writing non-blocking code, especially for I/O-bound operations (file access, network calls, database queries).

- **async keyword:** Marks a method as asynchronous.
 - **await keyword:** Suspends execution until the awaited task completes, without blocking the thread.
 - **Example flow:**
 - Without async/await: A file read operation blocks the main thread, freezing the UI.
 - With async/await: The file is read asynchronously, freeing the main thread to continue other work (like updating UI).
 - **Key Points:**
 - Asynchronous $\neq$ Parallel (async is about responsiveness, not about using multiple cores).
 - Often used in UI apps, web APIs, and high-performance servers.
 - Helps avoid deadlocks and improves scalability.
-

- **Concurrency** = “Juggling many balls” (tasks managed together).
- **Parallelism** = “Many jugglers juggling at the same time” (tasks executed at once).
- **Unit Testing/TDD** = “Check each Lego block before building the castle.”
- **Async/Await** = “Set a timer for the oven and do other chores while waiting, instead of staring at it.”